

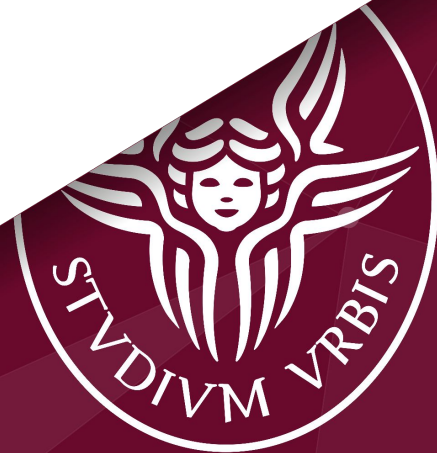


SAPIENZA
UNIVERSITÀ DI ROMA

Laboratorio di Ingegneria Informatica

Presentazione Esonero

Prof. Roberto Navigli
Dott. Domenico Fedele



Progetto

Obiettivo complessivo

- Costruire una **pipeline end-to-end** per **acquisire e analizzare documenti** da sorgenti **web eterogenee**
- **Applicazione reale**: il vostro lavoro contribuirà direttamente alla versione chatbot dell'LLM nazionale **Minerva** (sviluppato da **Sapienza NLP** e **Babelscape**)
 - Stiamo per lanciare una versione che cerca sul Web e recupera i risultati dal motore di ricerca, li parse e li utilizza per dare una risposta aggiornata
- E' una bella occasione per lavorare su un progetto concreto nel campo dell'Intelligenza Artificiale, con un impatto reale e immediato



Progetto

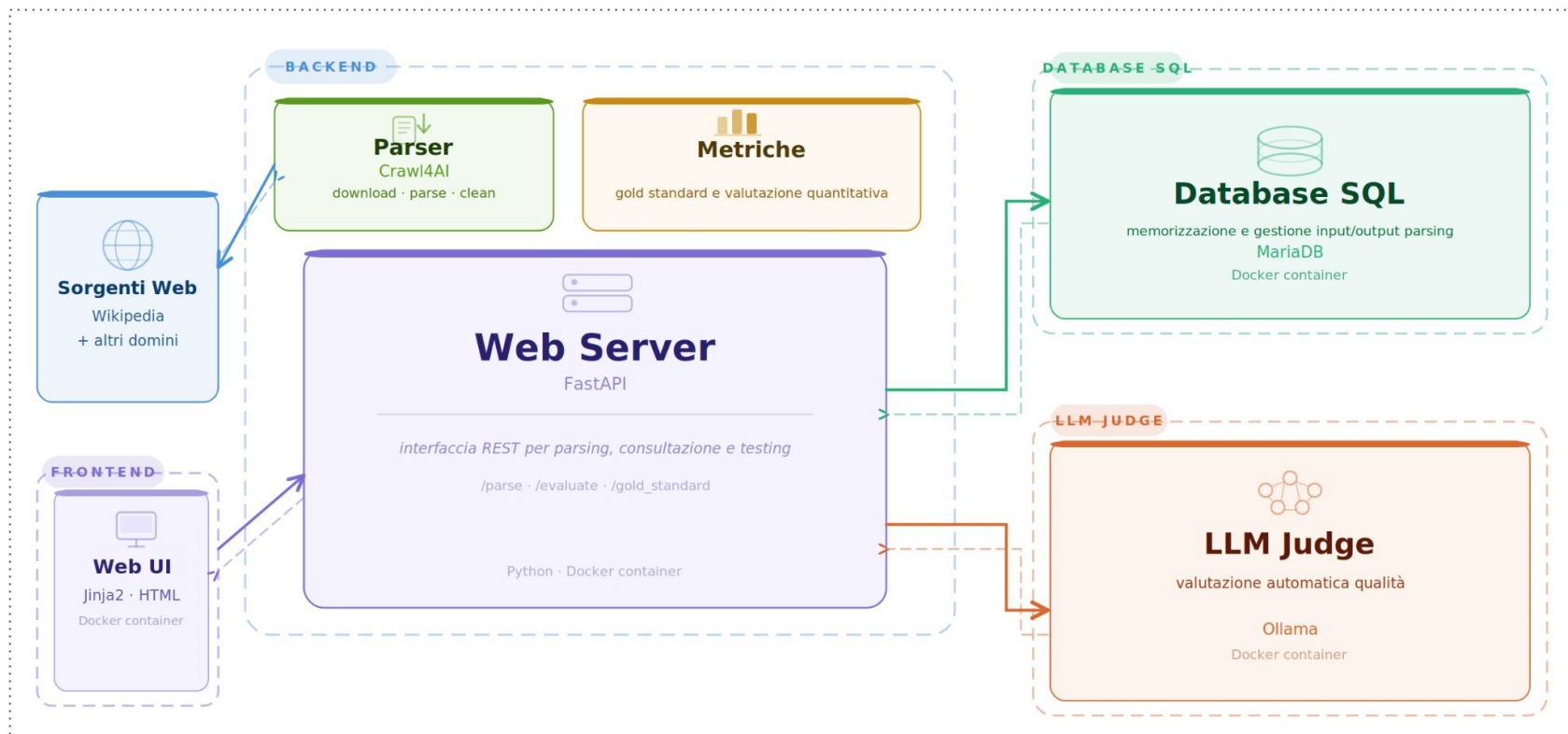
Funzionalità del sistema

- **Acquisizione di documenti** da sorgenti **web** eterogenee
- **Estrazione del testo pulito** dai documenti acquisiti tramite **parsing automatico**
- **Archiviazione** strutturata dei documenti in un **database relazionale**
- **Valutazione automatica** della **qualità** del testo
- **Giudizio qualitativo** aggiuntivo tramite un **LLM**
- Interfaccia **REST API** e **Web UI** per la consultazione, il testing e la gestione della pipeline
- **Architettura multi-container** orchestrata con Docker Compose (backend, frontend, database, LLM)



Progetto

Architettura del sistema



Progetto

Formazione dei gruppi

- Gli **esoneri e/o progetto finale** si svolgono in **gruppi da 3 persone** (devono essere mantenuti gli stessi gruppi)
- Entro il **9 aprile** è necessario compilare il **form** su Classroom per formalizzare il gruppo - **un gruppo un solo invio!!!**
- Chi ha **difficoltà a trovare un gruppo**, **si presenti il 9 aprile in aula** per un matchmaking.
- Eventuali richieste di **gruppi** con numero **ridotto** di studenti, **ben motivate ed eccezionali**, saranno valutate caso per caso (non implicano una semplificazione del progetto)

Esoneri

Informazioni generali

- **Consegna:** la consegna per l'esonero 1 è prevista il giorno 27 aprile 2026
- Il primo esonero viene assegnato oggi. Il secondo esonero concluderà il progetto, con consegna al massimo 8 giorni prima di ciascun appello (ENTRO SETTEMBRE)
- **Gli esoneri non sono obbligatori**
 - E' possibile consegnare il progetto completo in un'unica soluzione al massimo 8 giorni prima di ciascun appello
 - **Esempio:** appello il 25 dicembre → consegna entro il 17 dicembre (mezzanotte circa)
- **Accesso alla tesi**
 - Tutti gli studenti possono utilizzare il progetto svolto come tesi triennale, applicando una personalizzazione che sarà illustrata nelle prossime settimane
 - I migliori ~10 studenti potranno accedere a un **esonero 2 e progetto di tesi personalizzato in NLP**
- **Perché fare l'esonero?** Perché copre una parte sostanziale del lavoro progettuale, è seguito con attenzione dai docenti e offre una base già pronta per la prova finale



Esonero 1



Esonero 1

5 obiettivi

1. **Implementazione iniziale dei parser per i domini web assegnati**
Parser dedicati per estrarre il testo rilevante dalle pagine dei domini assegnati
2. **Costruzione di un gold standard (GS)**
Dataset di riferimento costruito manualmente, contenente il testo corretto da confrontare con l'output del parser
3. **Valutazione automatica dei parser con metriche specifiche**
Misurazione della qualità del parsing tramite metriche quantitative
4. **Esposizione dei dati via API REST**
Endpoint per parsing, accesso ai dati e valutazione automatica
5. **Web UI e Containerizzazione con Docker**
Interfaccia di test e ambiente eseguibile in modo semplice e uniforme

NOTA: Database e LLM as a Judge fanno parte dell'architettura complessiva del progetto, non sono richiesti in questo esonero



Obiettivo 1: Parser per Domini Web

Task

- **Parser:** implementare un parser Python specifico per le pagine di ciascun dominio web assegnato (es. it.wikipedia.org)
- Dato un URL proveniente da uno dei **domini assegnati** il parser deve **estrarre** le seguenti **informazioni**:
 - "url": "https//...",
 - "domain": "esempio.it",
 - "title": "Titolo della pagina web",
 - "html_text": "Testo estratto della pagina web grezzo (HTML)"
 - "parsed_text": "Testo estratto pulito senza tag HTML o di altro genere, in formato [MARKDOWN](#)"

Obiettivo 1: Parser per Domini Web

Libreria per il parsing - Crawl4AI

- Per il **download** e il **parsing** delle **pagine web** potete utilizzare la libreria [Crawl4AI](#) con **configurazione personalizzata** per ogni dominio
- **Crawl4AI** è una **libreria Python open-source** progettata per il **web scraping** orientato all'AI
- Trasforma qualsiasi **pagina web** in dati **strutturati** (**Markdown**, **JSON**, **screenshot**, **HTML**) **pronti** per essere consumati da **modelli linguistici** (LLM)
- Sfruttate la libreria per estrarre direttamente il **Markdown** ed eventualmente **ripulirlo** ulteriormente

Obiettivo 1: Parser per Domini Web

Crawl4AI - Installazione

- Installare la libreria
pip install -U crawl4ai
- Setup del browser (Chromium scaricato automaticamente)
crawl4ai-setup
- Verificare l'installazione
 - **crawl4ai-doctor**
 - **python -c "import crawl4ai; print('OK')"**
- Per eventuali problematiche potete consultare la [documentazione](#)

Obiettivo 1: Parser per Domini Web

Crawl4AI - Esempio di download e parsing

- [BrowserConfig](#)

Configura il browser
(headless, proxy, viewport)

- [CrawlerRunConfig](#)

Configura la singola richiesta. Agire sulla configurazione per migliorare l'output del parsing della libreria

- [AsyncWebCrawler](#)

Il motore del crawler; usato come context manager apre e chiude il browser in modo sicuro. `.arun()` visita l'URL e restituisce il contenuto estratto usando la config scelta. Dovete agire anche sull'output del parsing per migliorare la qualità.

```
import asyncio
from crawl4ai import AsyncWebCrawler, BrowserConfig, CrawlerRunConfig, CacheMode

async def main():

    # configura il browser (headless = senza finestra visibile)
    browser_cfg = BrowserConfig(headless=False)

    # configura la richiesta (BYPASS = scarica sempre dalla rete, ignora cache)
    crawler_cfg = CrawlerRunConfig(cache_mode=CacheMode.BYPASS)

    # apre il browser e lo chiude automaticamente alla fine del blocco
    async with AsyncWebCrawler(config=browser_cfg) as crawler:

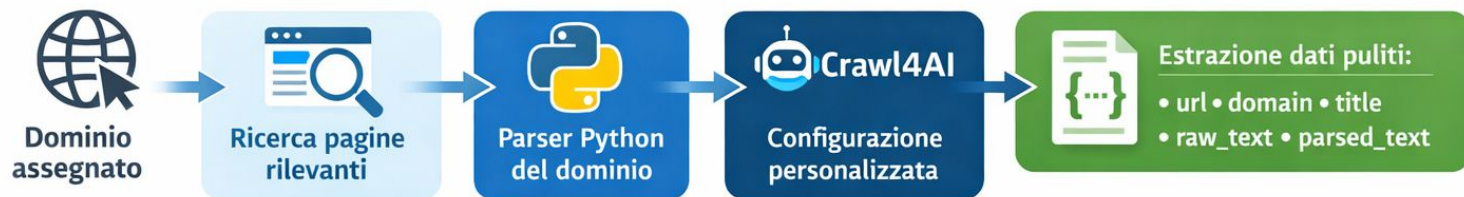
        # visita la pagina e aspetta che il crawl sia completato
        result = await crawler.arun(
            url="https://it.wikipedia.org/wiki/Minerva",
            config=crawler_cfg
        )

        # result.success          -> True se il crawl è andato a buon fine
        # result.error_message   -> messaggio d'errore in caso di fallimento
        #
        # alcuni output disponibili:
        # result.markdown         -> testo in Markdown, pronto per LLM e RAG
        # result.cleaned_html     -> HTML ripulito da script, stili e rumore
        # result.html             -> HTML completo della pagina (non ripulito)
        print(result.markdown)

# avvia il programma asincrono
asyncio.run(main())
```

Obiettivo 1: Parser per Domini Web

Pipeline di Parsing



1. **Selezione** manuale delle **pagine** più **rilevanti** del **dominio** assegnato
2. **Parser Python** dedicato per ciascun dominio
3. **Crawl4AI** con configurazione personalizzata per il dominio specifico (modificare **CrawlerRunConfig** e/o l'output del parsing)
4. Estrazione di **metadati** e **testo pulito** dall'output di Crawl4AI
5. **Output** strutturato: **url, domain, title, html_text, parsed_text**

Assegnazione dei domini da parsare

- Ogni gruppo dovrà **parsare pagine** provenienti da **domini web specifici** (es. it.wikipedia.org , <https://www.reddit.com>, <https://stackoverflow.com> e altri)
 - *NOTA: non dovete parsare la home page del sito, ma solo le pagine informative*
- Il **parsing** delle pagine provenienti dal dominio wikipedia.org è uguale e **obbligatorio** per tutti (in [inglese](#) se, ordinando i cognomi del gruppo in ordine alfabetico, il primo cognome inizia con una lettera “dispari”, ovvero A, C, E, G, I, K, M, O, Q, S, U, W, Y; in [italiano](#) altrimenti)
- **3 domini aggiuntivi** saranno assegnati **randomicamente** ad ogni gruppo:
 - Almeno **1 su 3** dei domini **aggiuntivi** vanno parsati correttamente per l'esonero
 - **Più domini** riuscirete a parsare correttamente **maggiore** sarà la **valutazione**
 - I **domini aggiuntivi** saranno **assegnati** alla **chiusura** del **form** per la formazione dei **gruppi** (dopo il 9 aprile, potete comunque cominciare con it.wikipedia.org o en.wikipedia.org)
 - Saranno comunicati su **Google Classroom**

Obiettivo 2: Gold Standard per Domini Assegnati

Cos'è un Gold Standard?

- Un **Gold Standard (GS)** è un insieme di **dati** di riferimento, costruito e verificato **manualmente**, considerato come **corretto**
- Serve a **valutare** la **qualità** dei risultati **prodotti** da un sistema automatico, fornendo un termine di confronto affidabile
- Si **confronta** l'**output del sistema** con il **GS**, così da misurare in modo oggettivo quanto il risultato prodotto sia corretto

Obiettivo 2: Gold Standard per Domini Assegnati

Gold Standard per il parsing di pagine Web

- **Gold Standard (GS):** per ogni **dominio assegnato**, selezionare almeno 5 URL rappresentative (pagine di tipo diverso, con contenuti vari, no home page)
- **Per ogni URL:** aprire la pagina nel **browser**, copiare **manualmente** il solo **testo informativo** (no pubblicità, contenuti collaterali o ripetuti in ogni pagina, ecc.)
 - **Includere:** titolo e corpo dell'articolo
 - **NON includere:** menu di navigazione, footer, sidebar, banner pubblicitari
- Utilizzare il **GS** per valutare la **qualità** dei vostri **parser**

Obiettivo 2: Gold Standard per Domini Assegnati

Formato del Gold Standard

- Salvare i **GS** in file JSON, uno per ogni dominio
- Un **entry** del vostro **GS** contiene le seguenti informazioni:
 - "url": "https//...",
 - "domain": "esempio.it",
 - "title": "Titolo della pagina web",
 - "html_text": "Testo estratto della pagina web grezzo (HTML)"
 - "gold_text": "Testo estratto pulito senza tag HTML o di altro genere"

Obiettivo 2: Gold Standard

Esempio Estrazione Info - Wikipedia

Indice

nascondi

Inizio

Titoli e ruoli

Cultura

Culto

Iconografia

Calculus Minervae

NOTE

Voci correlate

Altri progetti

Collegamenti esterni

WIKIPEDIA

25 anni di enciclopedia libera

Q

Cerca in Wikipedia

Ricerca

Minerva

79 lingue

Voce

Discussione

Leggi

Modifica

Modifica wikitesto

Cronologia

Strumenti

30 marzo: Wikipedia non può ancora essere venduta

Ci dispiace per i ripetuti tentativi, ma oggi è lunedì, 30 marzo e abbiamo ancora bisogno di aiuto. Se tutti coloro che leggono questo messaggio donassero oggi 2,75 € raggiungeremmo il nostro obiettivo subito. La maggior parte non dona. Wikipedia è gestita da un'organizzazione non-profit, non da un miliardario. Quindi se Wikipedia ti ha dato una conoscenza che vale almeno 2,75 €, per favore dona. Ogni contributo è utile, che siano 2,75 € o 25 €.

Dona 2,75 €

Dona un altro importo

FORSE DOPO

HO GIÀ DONATO

CHIUDI

Disambiguazione – Se stai cercando altri significati, vedi *Minerva (disambigua)*.

Minerva

(in latino *Minerva*) è la divinità romana della lealtà in lotta, delle virtù eroiche, della guerra giusta (guerra per giuste cause o per difesa), della *saggezza*, delle strategie, ed è riconosciuta anche protettrice degli artigiani.

Le sue origini discendono dagli Etruschi, che molto presto la fusero con *Athena*, suo corrispettivo nella *mitologia greca*. Minerva era venerata anche dagli antichi italici, come dimostra il santuario arcaico di *Lavinium*.

Come per *Athena* anche per *Minerva* l'animale sacro è la *civetta*, talvolta il gufo. Secondo il mito, era figlia di *Giove*, nata dalla testa di quest'ultimo.

Titoli e ruoli

[modifica | modifica wikitesto]

Il termine *Minerva* fu probabilmente importato dagli Etruschi che la chiamavano *Menrva*. I Romani ne confusero il nome straniero con il loro lemma *mens* (mente), dal momento che la dea governava non solo la guerra, ma anche le attività intellettuali

Minerva era figlia di Giove e, dopo l'assimilazione con la greca *Athena*, il mito della sua nascita la fece frutto dell'unione del re degli dei con la ninfa *Meti*.

Minerva, dettaglio del *Trionfo della Virtù* di Andrea Mantegna (1494-1502)

Fai una donazione

Registrati

Aspetto

nascondi

Modalità completano (baby Glott)

Disabilitata

Abilitata

Scopri di più sulla Modalità completano

Testo

Piccolo

Standard

Grande

Larghezza

Standard

Largo

Colore (beta)

Automatico

Chiaro

Scuri

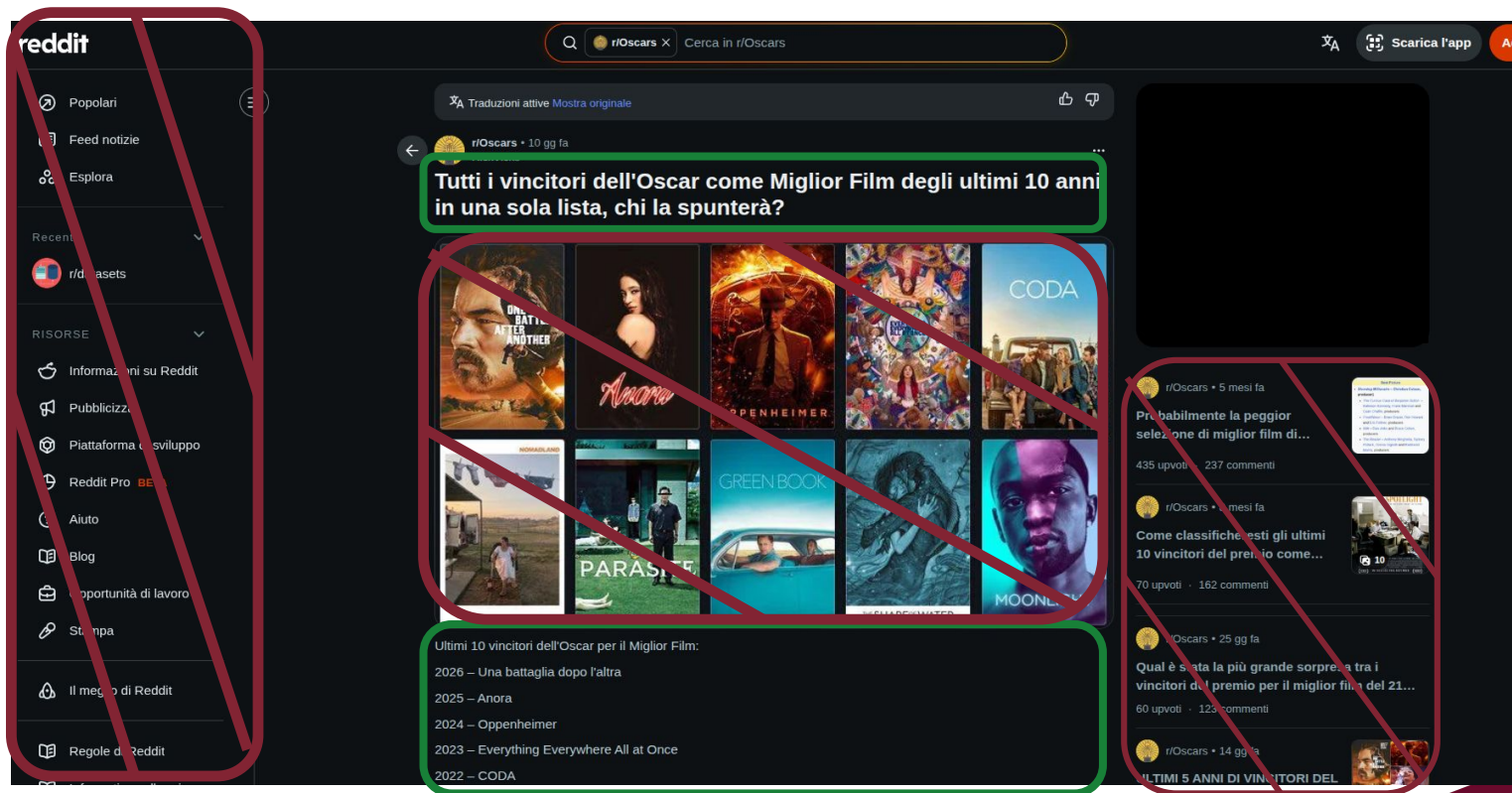
Navigli, Fedele - Laboratorio di Ingegneria Informatica



SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 2: Gold Standard

Esempio Estrazione Info - reddit



Obiettivo 3: Valutazione Parser

Valutare Qualità Parsing

- **Valutare la qualità dell'output** prodotto dai **parser** implementati
- La **valutazione** sarà effettuata tramite **metriche quantitative implementate da voi**, usate come **indicatori** oggettivi della **qualità** del parsing
- I **Gold Standard (GS)** prodotti saranno **utilizzati** come **dataset** di **test** per il confronto con l'output del parser
- Solo una **valutazione è obbligatoria** (**token_level_eval**), ma sono **apprezzate** anche eventuali **altre tecniche** di valutazione (sperimentate, cercate online, chiedete agli LLM!)

Obiettivo 3: Valutazione Parser

Valutare Qualità Parsing

- **Evaluation obbligatoria** da implementare “**token_level_eval**”:
 - La valutazione avviene a **livello di token** (parole separate per spazio in lowercase)
 - **tokens_estratti** = insieme dei token nel testo prodotto dal parser
 - **tokens_gs** = insieme dei token nel testo del Gold Standard
 - **Metriche**
 - **Precision** = $|tokens_estratti \cap tokens_gs| / |tokens_estratti|$
Tra i token trovati dal modello, quanti sono davvero corretti?
 - **Recall** = $|tokens_estratti \cap tokens_gs| / |tokens_gs|$
Tra tutti i token corretti possibili, quanti il modello è riuscito a trovare?
 - **F1** = $2 \times precision \times recall / (precision + recall)$
Media armonica tra precision e recall, utile per avere un unico indicatore bilanciato

Obiettivo 4: API REST

Overview API REST

- Implementare con **FastAPI** un **WEB Server** per accedere ai seguenti endpoint:
 - **GET /parse:** esegue il parser per un documento di un dominio
 - **GET /domains:** restituisce la lista dei domini assegnati
 - **GET /gold_standard:** restituisce il gold standard per un documento
 - **GET /full_gold_standard:** restituisce tutto il GS del dominio
 - **POST /evaluate:** dato risultato parsing e gs restituisce le metriche di evaluation
 - **GET /full_gs_eval:** restituisce l'evaluation aggregata su tutto il GS
- Usare **Pydantic** per validazione I/O
- I **formati di input/output** devono **rispettare** esattamente la **specifica**, perché saranno **testati automaticamente**
- **Gestire** esplicitamente **errori** come dominio non supportato e URL irraggiungibile

Obiettivo 4: API REST

GET /parse

- Seleziona automaticamente il parser corretto in base al dominio dell'URL ed esegue il parsing
- **Input:** "url": str (tramite url)
- **Output:** oggetto JSON nel formato indicato
- **Errori:**
 - Se il dominio non è supportato
 - Se l'URL è irraggiungibile

// OUTPUT - JSON

```
{  
  "url": "https://...",  
  "domain": "esempio.it",  
  "title": "Titolo estratto dalla pagina web",  
  "html_text": "Html estratto dalla  
                pagina web, input del parser",  
  "parsed_text": "Testo pulito, output del  
                parser di riferimento"  
}
```

Obiettivo 4: API REST

GET /domains

- Restituisce la lista dei domini supportati dal sistema (es. it.wikipedia.org)
- **Input:** nessuno
- **Output:** oggetto JSON nel formato indicato

// OUTPUT - JSON

```
{  
  "domains": [  
    "esempio_1.it",  
    "esempio_2.it",  
    ...  
  ]  
}
```

Obiettivo 4: API REST

GET /gold_standard

- Restituisce l'entry del GS per l'URL dato in input.
- **Input:** "url": str (tramite url)
- **Output:** oggetto JSON nel formato indicato.
- **Errori:**
 - Se il dominio non è supportato
 - L'URL non è nel GS.

// OUTPUT - JSON

```
{  
  "url": "https://...",  
  "domain": "esempio.it",  
  "title": "Titolo estratto dalla pagina web",  
  "html_text": "Html estratto dalla  
                pagina web, input del parser",  
  "gold_text": "Testo pulito, estratto  
                manualmente dalla pagina web"  
}
```

Obiettivo 4: API REST

GET /full_gold_standard

- Restituisce la lista degli elementi di un GS per un dominio specifico
- **Input:** "domain": str (tramite url)
(lo stesso usato nella lista get domains)
- **Output:** oggetto JSON nel formato indicato.
- **Errori:**
 - Se il dominio non è supportato.

// OUTPUT - JSON

```
{
  "gold_standard":
  [
    {
      "url": "https://...",
      "domain": "esempio.it",
      "title": "Titolo estratto",
      "html_text": "Html estratto",
      "gold_text": "Testo estratto a mano."
    },
    ...
  ]
}
```

Obiettivo 4: API REST

POST /evaluate

- Dato il risultato del parsing di un documento e il corrispettivo testo del GS restituisce le metriche di evaluation
- **Input:** {"parsed_text": str, "gold_text": str}
- **Output:** oggetto JSON nel formato indicato. Attenzione: **token_level_eval** con **precision**, **recall** e **f1** è obbligatorio

// OUTPUT - JSON

```
{
  "token_level_eval": {
    "precision": 0.87,
    "recall": 0.91,
    "f1": 0.89,
  },
  "x_eval": {
  }
}
```

Obiettivo 4: API REST

GET /full_gs_eval

- Restituisce la valutazione complessiva del Gold Standard per il dominio dato in input. Esegue il parsing di tutte le sue entry e aggrega i risultati tramite media delle metriche, calcolate sui singoli elementi del GS
- **Input:** "domain": str (tramite url)
- **Output:** oggetto JSON nel formato indicato.
- **Errori:**
 - Se il dominio non è supportato.

// OUTPUT - JSON

```
{
  "token_level_eval": {
    "precision": 0.87,
    "recall": 0.91,
    "f1": 0.89,
  },
  "x_eval": {
  }
}
```

Obiettivo 5: Web UI e Containerizzazione

Web UI

- **Implementazione della Web UI**

Realizzare l'interfaccia web in **HTML**, utilizzando **Jinja2** per il rendering dinamico dei contenuti.

- **Funzionalità principali**

- Consentire l'inserimento di un URL per mostrare un **confronto** tra testo della **pagina web grezzo** (HTML) e **testo pulito** senza tag HTML o di altro genere
- Mettere a disposizione un **menù a tendina** con tutti gli URL presenti nel **Gold Standard** (GS), in modo da eseguire rapidamente la UI
- Se per l'URL selezionato è disponibile il **GS**:
 - **confrontare** il risultato con il testo pulito presente nel GS
 - **mostrare la valutazione** tramite le **metriche** già implementate.

Obiettivo 5: Web UI e Containerizzazione

Containerizzazione con Docker

- **Containerizzazione** del sistema con **Docker** (vedi lezione del 9 aprile)
- **Requisiti**
 - **Backend** containerizzato (FastAPI) sulla porta **8003**
 - **Frontend** containerizzato (HTML + Jinja2)
 - **docker-compose.yaml** per avviare tutto con un solo comando (come visto a lezione)
 - Usare solo path relativi (nessun path assoluto)
 - Deve funzionare su Ubuntu subito dopo:
docker compose up --build

Struttura delle Directory del Progetto

- **docker-compose.yaml**: avvia l'intero sistema con un solo comando
- **backend/**: server FastAPI, parser, modulo di valutazione
- **frontend/**: interfaccia web HTML/Jinja2 con template
- **gs_data/**: Gold Standard JSON per ciascun dominio
- **domains.json**: lista dei domini assegnati
- **report.pdf**: max 2 pagine, redatto in LaTeX

```
progetto/  
├── docker-compose.yaml  
├── domains.json  
├── gs_data/  
│   ├── dominio_x_gs.json  
│   └── dominio_y_gs.json  
├── backend/  
│   ├── Dockerfile  
│   ├── requirements.txt  
│   └── src/  
│       └── server.py  
├── frontend/  
│   ├── Dockerfile  
│   ├── requirements.txt  
│   └── src/  
│       ├── frontend.py  
│       └── templates/  
└── report.pdf
```

Test automatico

Test del backend

- Vi forniremo nei prossimi giorni su Google Classroom uno **script** che avvierà un **test automatico** di tutti gli **endpoint** definiti nella specifica del **WEB Server** del progetto
- Dovete **esporre la backend** del vostro **progetto** sulla porta **8003**. Lo script di testing, eseguito fuori dai Docker container del vostro progetto, deve poter accedere a quella porta per eseguire il test automatico (non modificare la porta nello script di test)
- Il test deve funzionare subito dopo il comando: **docker compose up --build**
Non ci devono essere passaggi intermedi tra questa inizializzazione automatica e l'esecuzione dello script di test



Test automatico

Test del backend

- Dovete riuscire ad **eseguire** l'intero **script di test** senza modifiche arrivando a vedere:
"ALL TESTS PASS!"
Indispensabile prima di effettuare la consegna
- **Non** dovete **modificare** lo **script di test**, dato che noi lo useremo esattamente così com'è per valutare i progetti
- Se pensate di aver **trovato un errore** nello script di test **comunicarlo** tramite **Google Classroom**



Test automatico

Test del frontend

- Ricordate di **testare** voi stessi anche il **frontend** del vostro **progetto**, dopo aver eseguito i test automatici con successo e prima di consegnare
- Il frontend **non viene valutato automaticamente** dallo script di test
- **Non** dovete **consegnare** lo **script di test** insieme al vostro progetto (è uguale per tutti)



Test automatico

Test finali prima della consegna

- Il comando **docker compose up --build** deve funzionare su un qualsiasi computer, quindi non si possono usare path assoluti ma solo path relativi.
Prima di consegnare spostate il vostro progetto in un'altra cartella nel vostro computer e controllare che **docker compose up --build** funzioni ancora ***senza cambiare nulla*** per assicurarvi che funzioni anche sul nostro computer che useremo per le correzioni.
- Potete sviluppare su Mac/Windows ma noi eseguiremo il comando **docker compose up --build** su Linux (Ubuntu), quindi se state sviluppando su Mac/Windows dovete controllare che questo comando funzioni correttamente anche su **Ubuntu** prima di consegnare (usate un **macchina virtuale**, vedere slide successive)
- Ricordate di eseguire anche il test automatico e di testare il frontend dopo ognuno di questi test



Consegna



Consegna

Singolo Zip

- Consegnare un **singolo file zip** che è lo zip della **cartella** del vostro **progetto**, con il nome che include tutte le matricole dei membri del gruppo:
matricola1_matricola2_matricola3_lab_esonero_1.zip
- La **struttura** delle **directory** all'interno del file zip deve essere in linea con quella **specificata** nelle **slide precedenti**
- Potete usare più file, cartelle e sottocartelle se li repute utili a organizzare meglio il progetto



Consegna

Docker Compose

- Il vostro **progetto** verrà **eseguito** tramite il comando **docker compose up --build** eseguito da terminale nella directory del file **docker-compose.yaml**
- Il file **docker-compose.yaml** contiene le istruzioni necessarie per avviare l'intero progetto con il comando **docker compose up --build**
- I due moduli **frontend** e **backend** sono separati in cartelle diverse, ognuna con i suoi file **requirements.txt** e **Dockerfile**, necessari per la costruzione del loro Docker container. Nel modulo **frontend** c'è la cartella **templates** per i file HTML necessari per creare l'interfaccia web.

Consegna

Step da Seguire (1)

- Scaricate l'ISO di **Ubuntu 22.04.5** (desktop) da <https://releases.ubuntu.com/jammy/>
- Installarlo o come **macchina virtuale** o come **dual boot**. Fate il test finale in un nuovo dual boot o in nuova macchina virtuale, non nel PC in cui avete sviluppato il progetto, questo vi garantisce che funzionerà anche nel mio.
- Tutte le **istruzioni** seguenti vanno **eseguite** dentro la **macchina virtuale** o il **dual boot**.
- Installate **docker** e **docker compose** con le versioni corrette (vedi Google Classroom e form di consegna).
- Navigate da terminale nella **cartella** del vostro **esonero** esattamente come la consegnerete, salvatene anche una copia dato che la dovete consegnare come è prima di **eseguire i test** e non dopo



Consegna

Step da Seguire (2)

- Eseguire il comando “**sudo chmod 777 -R .**” per dare i permessi necessari e potenzialmente altre cartelle necessarie.
- Eseguire il comando “**sudo docker compose up --build**” e aspettare che l’inizializzazione termini
- Eseguire il comando lo script di test che deve funzionare dando in output “**ALL TESTS PASS**”
- Consegnare la copia della cartella così come era prima di eseguire il test compressa in un unico file **.zip**



Consegna

Report

- Realizzare un **report** in un file **report.pdf** (anche questo incluso nell'archivio zip) di massimo **2 pagine** (fontsize 11) che descrive il vostro progetto:
 - **Obiettivo del progetto:** Brevissimo riassunto della funzionalità del progetto (1-2 frasi)
 - **Organizzazione del codice:** Come è strutturato il vostro codice e una spiegazione della (singola) responsabilità di ogni modulo
 - **Valutazione parser:** Come sono stati valutati i parser e i risultati ottenuti
 - Siete liberi di aggiungere anche altre sezioni (all'interno del limite totale di due pagine)
 - Il report può essere anche più breve di due pagine (apprezziamo la sintesi)
- [Il report deve essere scritto in Latex partendo da questo template online](#)
 - Non includere l'abstract, partite direttamente con la prima **`\begin{section}`**
 - Decomentare **`\aclfinalcopy`** eliminando il % davanti a quella riga del file **.tex** per poter mostrare il vostro nome e cognome (altrimenti verrà mostrato "anonymous submission")
- Il progetto va consegnato su **Google Classroom** (tramite l'apposito compito/form)



Valutazione

- L'esonero conterà metà del progetto
- Saranno valutati:
 - L'effettivo funzionamento del progetto (bloccante: non si passa se il progetto non funziona e non passa il test automatico)
 - L'organizzazione del progetto (cartelle, file, ecc.)
 - La qualità e la pulizia del codice (inclusi i commenti e i type hint)
 - La modularità del progetto: la corretta suddivisione in moduli
 - La corretta gestione dei casi limiti o di errore
 - Il report
- Il plagio (sia chi fa copiare che chi copia) porterà all'annullamento del progetto